Code Review and Security Audit

Custom Thumbnail Generator - WordPress plugin, version 1.1
Date: 23 October 2026 · Reviewer: Claude (AI code review, requested by the plugin developer) · Environment: WordPress 7.1.x, PHP 7.4 (Lando), Imagick + GD

Verdict. No high-severity vulnerability was found: every AJAX action is protected by a nonce and the `manage_options` capability, every database query is prepared, no endpoint accepts a file path, and all output is escaped. The review confirmed **9 security issues (1 medium, 7 low, 1 informational)** and **5 functional defects**; all were fixed and each fix was verified by a test that failed before the fix and passes after it. Six further items are recommendations that were deliberately not changed (section 7). PHPCS (WordPress Coding Standards), PHPCompatibilityWP (PHP 7.0+) and Plugin Check report no findings on the final code.

0	1	8	5
High / critical	Medium (fixed)	Low / info (fixed)	Defects (fixed)

1. Scope and method

Code reviewed (all shipped source, read in full):

Area	Files	Lines
Bootstrap, uninstall	`custom-thumbnail-generator.php`, `uninstall.php`	72
Settings page and legacy features	`includes/init.php`, `includes/settings-page.php`, `includes/class-ctgen-functions.php`, `includes/class-ctgen-ajax-handlers.php`, `templates/ctgen-form-html.php`, `templates/ctgen-custom-thumb-list-html.php`	983
WebP conversion	`webp/class-ctgen-webp-converter.php`, `webp/class-ctgen-webp-ajax-handlers.php`	986
Attachment Summary	`includes/class-ctgen-usage-scanner.php`, `includes/class-ctgen-attachment-summary.php`, `includes/class-ctgen-attachment-summary-table.php`	1,544
JavaScript	`assets/js/ctgen-webp-convert.js`, `assets/js/ctgen-attachment-summary.js`, `assets/js/ctgen-media-actions.min.js` (shipped minified only; reviewed after un-minifying)	1,229
CSS	`assets/css/*.css` (checked for external URLs: none, fonts are local)	373

Method.

1. Manual line-by-line review of every request boundary (12 AJAX actions, list-table query arguments, screen option), every database query, every filesystem write/delete, every HTML output and every JavaScript DOM insertion.
2. Automated standards checks: PHPCS with the WordPress ruleset (`.phpcs.xml.dist`), PHPCompatibilityWP for PHP 7.0 and later, Plugin Check on the release build, `php -l` on every PHP file, `node --check` and source/minified drift check on every script.

3. Every suspected issue was **reproduced on the running site before it was reported** with a probe written for this review (`v2/tests/cr-probe.php`, 14 checks; `v2/tests/cr-browser.js`, 6 checks in Chromium). Each check states the secure behaviour as its pass condition, so a failure is a confirmed defect. Suspicions that did not reproduce are not reported as findings (section 6).
4. After the fixes, the full regression suite of the plugin (M1-M4 and Attachment Summary suites) was run again to make sure nothing else changed.

All probes use disposable fixtures only; the legacy "regenerate" loops were restricted to the fixtures, or their AJAX responses were answered by the test, so no real media was processed. The database (options, posts, post meta) and the uploads folder were compared before and after every run and were identical.

2. Summary of findings

ID	Severity	Finding	Status
S1	MEDIUM	WebP copies of a deleted image stay on disk and publicly reachable	**Fixed**
S2	LOW	DOM-based XSS sink in the settings-page script (size name inserted as HTML)	**Fixed**
S3	LOW	No server-side validation of new custom sizes (0, negative, text, 99999 accepted)	**Fixed**
S4	LOW	Scripts, styles and the AJAX nonce loaded on every admin screen	**Fixed**
S5	LOW	Temporary WebP files of an interrupted conversion never cleaned up	**Fixed**
S6	LOW	Theme name and size names output with `wp_kses_post()` instead of `esc_html()`	**Fixed**
S7	LOW	Two PHP files without a direct-access guard	**Fixed**
S8	LOW	"Regenerate one image" accepted any post ID (no attachment/image check)	**Hardened**
S9	INFO	Legacy settings-page script shipped only minified (no human-readable source)	**Fixed**
B1	MEDIUM	"Regenerate one size" saved a full-size copy as that size when the image is smaller	**Fixed**
B2	LOW	"Regenerate Thumbnails" stops for good when one request fails	**Fixed**
B3	LOW	Every add-size error shown as "Thumbnail size exists!"	**Fixed**
B4	LOW	A damaged custom-size option causes PHP warnings on every request	**Fixed**
B5	LOW	Version constant defined only on `admin_init` (undefined-constant warnings elsewhere)	**Fixed**
W1-W6	STANDARDS	Untranslated strings, invalid table markup, relative include, typos, null dependencies (section 5)	**Fixed**

Severity reflects that every affected action already requires an administrator (`manage_options`) and a valid nonce; none of the findings is exploitable by a visitor or a lower-role user on its own.

3. Security findings

S1 · MEDIUM · WebP copies of a deleted image stay on disk and publicly reachable

Location	`webp/class-ctgen-webp-ajax-handlers.php` (no `delete_attachment` handling existed)
Issue	WebP copies are written next to the image (`photo.jpg` → `photo.webp`, `photo-150x150.webp`, ...). WordPress deletes only the files it knows about, so when an administrator permanently deleted an image, every WebP copy remained in the uploads folder.
Impact	An image deleted to take it off the site (personal data, a copyright complaint, a leaked picture) stays publicly downloadable at a predictable URL (same name, `.webp` extension). Also leaves orphaned files.
Evidence	Probe CR6 before the fix: *"7 WebP copies created; after Delete permanently: 7 left on disk and still publicly reachable"*.
Fix	New `delete_webp_copies()` on `delete_attachment` (runs before WordPress removes the image files). It deletes the copies of the main file, the original upload, every size and every pre-edit backup size, with `wp_delete_file()`. Deliberately conservative: JPEG/PNG attachments only; folder must be inside the uploads directory (`realpath` check); nothing is deleted when another image shares the base name; never a WebP that is a Media Library item of its own; only a valid WebP whose dimensions equal those of the file it was made from.
Verified	CR6 after: *"4 WebP copies created; after Delete permanently: none left"*. CR7: deleting `ctgen-cr-shared.jpg` keeps the separate Media Library item `ctgen-cr-shared.webp` byte-identical.

S2 · LOW · DOM-based XSS sink in the settings-page script

Location	`assets/js/ctgen-media-actions.min.js`, "regenerate size" handler: `$(cell).html(slug + ' <span ...>working...</span>')`
Issue	The size name read from the button's `data-slug` attribute was inserted into the page as HTML.
Impact	Script execution in an administrator's session if a size name ever contained markup. Sizes are created by the plugin as `ctgen_<w>x<h>`, so exploitation needs the option to be altered by other means (database access, another vulnerable plugin); hence LOW, but it is a real sink.
Evidence	CR-B2 against the original script, with `data-slug` set to `<img src=x onerror="window.__ctgenXss=1">`: *"<img> elements created: 1, script ran: true"*.
Fix	Added the readable source `assets/js/ctgen-media-actions.js` (built into the .min.js by `v2/tools/build-js.js`). All size names and server messages are inserted with `.text()`; the only remaining `.html()` is the server-rendered, escaped size table. Size names in the table are now printed with `esc_html()` (S6).
Verified	CR-B2 after: *"<img> elements created: 0, script ran: false"*; the name is shown as text.

S3 · LOW · No server-side validation of new custom sizes

Location	`includes/class-ctgen-ajax-handlers.php` lines 42-44 (original): `absint( sanitize_text_field( ... ) )`
Issue	Only the browser form had `min="25"`; the server accepted any value and silently converted it.
Impact	Malformed sizes registered for the whole site (applied to every future upload): `ctgen_0x0`, `-150` stored as `150`, text stored as `0`, and 99999 x 99999.
Evidence	CR2 before: *"0 x 0 -> ctgen_0x0; -150 x -150 -> ctgen_150x150; abc x 10 -> ctgen_0x10; 99999 x 99999 -> ctgen_99999x99999; 10 x 10 -> ctgen_10x10"*.
Fix	Digits only, 25 to 10,000 px (constants `MIN_DIMENSION` / `MAX_DIMENSION`), otherwise a translated error; crop stored as a boolean; the form fields got `max="10000" step="1"`. The stored option is now read through `ctgen_get_custom_sizes()`, which keeps only well-formed entries.
Verified	CR2 after: all five invalid inputs rejected; 123 x 77 with crop stored; duplicate rejected. CR-B1: the page shows the server's message.

S4 · LOW · Scripts, styles and the AJAX nonce loaded on every admin screen

Location	`includes/init.php` line 124 (original), `ctgen_enqueue_scripts()` without a page check
Issue / impact	The settings-page script with its `ctgen_media_actions` nonce, the admin CSS and Font Awesome 4.7 were printed on every admin screen, for every logged-in role. The nonce alone grants nothing (all actions also check `manage_options`), but it widens exposure without need, adds weight to every admin page and lets the global CSS/Font Awesome clash with other plugins. WordPress.org guidelines ask plugins to load assets only on their own screens.
Evidence	CR9 before: *"on Dashboard, Media and Posts screens: ctgen-media-actions-script, ctg-admin.css, ctg-fa-css"*.
Fix	Script, nonce and Font Awesome only on the Settings page; the shared admin CSS on Settings and Attachment Summary; nothing elsewhere.
Verified	CR9 after: *"nothing from this plugin"*; CR-B4 in Chromium: Settings has script + CSS + icons, Media Library has none, Attachment Summary has only the CSS (header renders).

S5 · LOW · Temporary files of an interrupted conversion never cleaned up

Location	`webp/class-ctgen-webp-converter.php` line 245 (original)
Issue / impact	Each conversion writes `.ctgen-webp-tmp-<random>.webp` and renames it into place. If PHP stopped in between (time or memory limit), the temp file stayed in the public uploads folder for good.
Fix	Before writing, the converter removes this plugin's temp files older than one hour from that folder (once per folder per request). Newer temp files (a conversion may still be running) and any other file are never touched.
Verified	CR12: old plugin temp file removed; fresh plugin temp file kept; unrelated old dot-file kept.

S6 · LOW · Text printed with `wp_kses_post()` instead of `esc_html()`

Location	`includes/class-ctgen-functions.php` lines 140-142; `templates/ctgen-custom-thumb-list-html.php` line 32 (original)
Issue / impact	The active theme's name and the custom size names were output through `wp_kses_post()`, which allows HTML; on line 142 the closing `</span>` was even outside the sanitised string. Theme names come from the theme's header, so any HTML in it was rendered on the settings page.
Fix	Both are plain text now: `esc_html()`; the "WordPress Core" label is translatable.

S7 · LOW · PHP files without a direct-access guard

Location	`includes/class-ctgen-ajax-handlers.php`, `includes/class-ctgen-functions.php`
Issue / impact	Both files only declare a class, so a direct request ran no logic; still, WordPress.org requires the guard on every PHP file, and it prevents future code in these files from running outside WordPress.
Fix / verified	`if ( ! defined( 'ABSPATH' ) ) { exit; }` added. CR8: all 13 shipped PHP files have the guard (`uninstall.php` uses `WP_UNINSTALL_PLUGIN`).

S8 · LOW · "Regenerate one image" accepted any post ID

Location	`includes/class-ctgen-ajax-handlers.php` lines 122-139 (original)
Issue	The ID was cast with `(int)` (no `wp_unslash`) and passed to `wp_generate_attachment_metadata()` without checking that it is an image attachment. The probe showed WordPress itself returned nothing for a plain post, so no data was damaged (CR5 passed before the fix).
Fix	Defence in depth: `absint( wp_unslash() )`, must be an image attachment with an existing file, otherwise a clear translated error.

S9 · INFO · Settings-page script shipped only minified

Issue	WordPress.org guideline 4 requires human-readable code; `ctgen-media-actions.min.js` had no source in the plugin, which also made it impossible to review or fix safely.
Fix	Readable `assets/js/ctgen-media-actions.js` added and shipped; the .min.js is built from it, and the drift check confirms they match. `window.reloadThumbList()` is kept for compatibility.

4. Functional defects

B1 · MEDIUM · "Regenerate one size" saved a full-size copy as that size

Location	`includes/class-ctgen-ajax-handlers.php` lines 199-219 (original)
Issue	The result of `$editor->resize()` was ignored. For an image smaller than the size, `resize()` fails, but `save()` still wrote an unresized copy and the plugin recorded it in the metadata as that size.
Evidence	CR4 before, 300 x 200 image and a 1200 x 900 size: *new file ctgen-cr-small-300x200.jpg; metadata for the size: {"file":"ctgen-cr-small-300x200.jpg","width":300,"height":200}; "1 ctgen_cr_big thumbnails generated successfully."*
Fix	Uses core's `make_subsize()`, which never upscales (as WordPress does on upload). `make_subsize()` is not part of the `WP_Image_Editor` base class, so other image editors fall back to `multi_resize()`, exactly as core's `_wp_make_subsizes()` does.
Verified	CR4: no file, nothing recorded. CR4a: a 120 x 80 size is created as `ctgen-cr-small-120x80.jpg` (same naming as before). CR4b: with a test editor without `make_subsize()`, the fallback creates the small size and refuses the large one, with no PHP error.

B2 · LOW · "Regenerate Thumbnails" stops for good when one request fails

Issue	The next image was requested only from the success callback; one failed request (server error, timeout) left the run stuck on "Processing..." with no message.
Evidence	CR-B3 against the original script (3 images, first answer HTTP 500): *"1 requests, statuses (none); button Processing..."*.
Fix / verified	Every request advances the run (`.always()`). CR-B3 after: 3 requests, "Processed 1 of 3 > Processed 2 of 3 > Thumbnail generation complete!", button restored.

B3 · LOW · Every add-size error shown as "Thumbnail size exists!"

Fix / verified	The server's message is shown (as text). CR-B1 before: "Thumbnail size exists!" for a 10 x 10 size; after: "Width and height must be whole numbers from 25 to 10000 pixels." The remove-size error also showed `undefined` (it read `response.message` instead of `response.data.message`); fixed.

B4 · LOW · A damaged custom-size option causes PHP warnings on every request

Location	`includes/init.php` `ctgen_register_custom_sizes()` and `ctgen_add_custom_sizes_to_media_dropdown()` (original)
Fix / verified	Both read the option through `ctgen_get_custom_sizes()` (array check, slug format, positive integers). CR10 before: *"Invalid argument supplied for foreach()"*; after: no warnings. The site's real stored size (`ctgen_250x250`) passes the check.

B5 · LOW · Version constant defined only on `admin_init`

Issue	`CTGEN_PLUGIN_VERSION` was defined from `get_plugin_data()` on every `admin_init` (a file read and header parse on each admin request); wherever `admin_init` had not run, using it raised "Use of undefined constant" warnings (observed during the CR9 probe).
Fix	Defined once at load: `define( 'CTGEN_PLUGIN_VERSION', '1.1' )`. The release script (`v2/deploy/svn-release.sh`) now refuses to build when it differs from the `Version:` header.

5. WordPress and PHP standards

ID	Finding	Change
W1	User-visible strings were hard-coded in English: AJAX messages in `class-ctgen-ajax-handlers.php`, the "template missing" error, "WordPress Core", and every dialog and status of the settings-page script.	All wrapped in `__()` / `esc_html__()` with translator comments; the script receives its strings through `wp_localize_script()`. English wording of the dialogs is unchanged. Translation template regenerated: 209 strings, 21 of them new.
W2	Invalid markup: the custom sizes table nested every `<tr>` inside another `<tr>`.	Outer row removed; screenshots of the table are pixel-identical before and after.
W3	Relative `require_once 'ctgen-custom-thumb-list-html.php'` (depends on PHP's include path).	`require_once __DIR__ . '/...'`.
W4	`null` passed as style dependencies; message typos ("Plase trye"); `esc_html()` misused inside `empty()`.	`array()`; typos corrected; plain boolean check.
W5	Five copies of the capability check with an untranslated, unescaped `wp_die()` message.	One `authorize()` helper with `esc_html__()`.
W6	Dependency on the optional PHP `ctype` extension avoided in the new validation.	Regular expression instead of `ctype_digit()`.

Automated results on the final code:

Check	Command	Result
WordPress Coding Standards	`phpcs --standard=.phpcs.xml.dist` (all 13 shipped PHP files)	**0 errors, 0 warnings**

PHP 7.0+ compatibility	`phpcs --standard=PHPCompatibilityWP --runtime-set testVersion 7.0-`	**0 findings**
Plugin Check	`lando wp plugin check` on the release build (`v2/deploy/svn-release.sh`)	**No errors found (release build, 38 files)**
PHP syntax	`php -l` on every PHP file of the plugin	**21/21 files clean**
JavaScript	`node --check; build-js.js --check` (each .min.js equals a fresh build of its source)	**3/3 OK**

6. Areas verified secure (no change needed)

Area	What was checked	Evidence
CSRF and authorisation	All 12 AJAX actions call `check_ajax_referer( 'ctgen_media_actions', 'security' )` and require `manage_options`; none is registered for logged-out users (`wp_ajax_nopriv_`).	CR1: forged nonce refused 12/12; subscriber *and* editor with a valid nonce refused 12/12; no nopriv handlers.
SQL injection	Usage scanner queries use `$wpdb->prepare()` with `esc_like()`; list-table filters, sort keys and order are allow-listed; meta queries use fixed keys.	Code review; A1 suite.
Path traversal / arbitrary file write or delete	No endpoint accepts a path: only an attachment ID and a size slug validated against registered sizes / the attachment's metadata. File names from metadata are reduced to `wp_basename()`. The converter writes only `.webp` files in the attachment's folder, refuses a destination that resolves to the source (including through `..` and symlinks), never overwrites another Media Library item, and writes through a verified temp file with an atomic rename.	M1 items 11-13, M2 items 17-24, A1 item 11.
Stored XSS	Attachment and post titles, custom field names, option names and file names are escaped on output (`esc_html`, `esc_attr`, `esc_url`).	CR3: titles `"><img src=x onerror=...><script>` and a field named `<b>field</b>` appear only as escaped text.
DOM XSS (new scripts)	`ctgen-webp-convert.js` and `ctgen-attachment-summary.js` insert all messages with `.text()`; HTML is only the server-escaped table cell, parsed with `$.parseHTML()` (scripts not executed); the image viewer sets `src` from an `esc_url()` link.	Code review; A1 browser suite.
Information disclosure	Error messages are generic and translated; responses contain file names only, never server paths.	M2 item 18 (37 responses scanned), A1 item 11.
Resource limits	One image per WebP request; usage scan 1-25 IDs per request; screen option bounded to 1-200 rows; 2-minute client timeout per request.	A1 item 5; M3 item 23.
Uninstall	Removes only the plugin's own scan meta and screen option; images, WebP copies and custom sizes are kept by design (documented).	A1 item 9.
Not reproduced (not a finding)	Suspected global leak of the `wp_allow_query_attachment_by_filename` filter: WordPress core removes it right after use (`WP_Query` calls `remove_all_filters()`).	CR11 passes; `wp-includes/class-wp-query.php` line 1932.

7. Recommendations not implemented

#	Recommendation	Why not changed now / residual risk

R1	"Regenerate one size" processes the whole library in a single request (`posts_per_page => -1`). Process it in batches like the WebP conversion.	Needs a new UI flow (feature work). Risk: on large libraries the request can hit the PHP time limit; work already done is kept, the rest is not processed. Administrator-only.
R2	Use one nonce action per AJAX action instead of the shared `ctgen_media_actions` .	All actions require `manage_options` , so a leaked nonce gives no extra power. Low value, touches every script.
R3	Replace Font Awesome 4.7 (2016; about 108 KB per page load for its CSS and WOFF2 font, and 1.1 MB of font files in the package) with Dashicons.	Visual change; now loaded only on the Settings page, so impact is small.
R4	Move `banner-772x250.jpg` , `screenshot-1.png` and the unused icons (64, 128, 256 px) from `assets/images/` to the SVN `/assets` folder.	Not referenced by any code (only `icon-80x80.png` is). Removing them from the release is a packaging decision for the developer.
R5	When a custom size is removed or an image is edited, older WebP copies of the old file names remain until "Regenerate WebP" or deletion of the image.	Matches the existing design ("existing thumbnails remain intact"); deletion of the image now removes them (S1).
R6	Consider a capability filter (for example `upload_files` for the read-only Attachment Summary).	Product decision; `manage_options` everywhere is the safest default.

8. Test evidence

Code-review probes: before and after the fixes

Check	Before	After
CR1 CSRF and authorisation, 12 actions	**PASS**	**PASS**
CR2 add-size server validation	**FAIL**	**PASS**
CR3 stored XSS via titles and field names	**PASS**	**PASS**
CR4 / CR4a / CR4b regenerate one size (no upscale; small size; fallback editor)	**FAIL / - / -**	**PASS / PASS / PASS**
CR5 regenerate one image accepts only images	**PASS**	**PASS**
CR6 WebP copies deleted with the image	**FAIL**	**PASS**
CR7 a WebP Media Library item is never deleted	**PASS**	**PASS**
CR8 direct-access guards	**FAIL**	**PASS**
CR9 assets only on plugin pages	**FAIL**	**PASS**
CR10 damaged option	**FAIL**	**PASS**
CR11 file-name search filter	**PASS**	**PASS**
CR12 stale temp files	(added with the fix)	**PASS**
CR-B1 / B2 / B3 browser: error message, XSS sink, stalled run	**FAIL / FAIL / FAIL**	**PASS / PASS / PASS**
CR-B4 / B5 / B6 browser: assets per page, JS errors, nothing changed	-	**PASS / PASS / PASS**

"Before" for CR1-CR11 = the original PHP code; for CR-B1 to CR-B3 = the original `ctgen-media-actions.min.js` served to the browser in place of the new one (`CTGEN_CR_OLD_JS`), log `v2/tests/results/cr-browser-old-script.txt` .

Full regression suite on the final code

Command: `v2/tests/m4-run-all.sh v2/tests/results/cr-regression` (started 23 October 2026, 14:09 UTC). Logs: `v2/tests/results/cr-regression/` .

Suite	Result
php -l, every PHP file	**21/21 clean**
PHPCS (WordPress ruleset), whole plugin	**exit 0**
JavaScript syntax and source/minified drift	**3 scripts OK**
M1 converter harness (formats, corrupt/missing files, collisions, retries, source preservation)	**20/20 PASS**
M2 backend harness (endpoints, nonce/capability, scopes, path leaks: 37 responses scanned, counts)	**26/26 PASS**

Empty media library (real; database restored, CHECKSUM TABLE identical)	**PASS**
M3 browser suite (WebP controls, progress, errors, keyboard, live region, shipped .min.js)	**18/18 PASS**
Attachment Summary harness / browser suite / performance (400 images)	**8/8 PASS / 21/21 PASS / 0 wrong counts, 135 ms per image**
Admin menu check	**9/9 PASS**
Code-review probes / browser checks	**14/14 PASS / 6/6 PASS**
Plugin Check, release build (38 files)	**No errors found**
No development tooling loaded or shipped	**none found**
Database fingerprint (options, posts, post meta) before/after; uploads folder	**identical; 256 files md5-identical; 0 temporary users**

Legacy UI comparison (regenerate/remove controls vs. the capture taken before the WebP work): dialogs, AJAX requests, the bulk regenerate sequence, row regenerate and row remove sequences are unchanged, and all screenshots are pixel-identical. Compared with the previous run, the only new difference is the hash of the settings-page script, which is expected: it is now built from the readable source.

9. Files changed

File	Change	Findings
`custom-thumbnail-generator.php`	Version constant defined at load	B5
`includes/init.php`	Assets only on plugin pages; localized script strings; `ctgen_get_custom_sizes()`	S4, B4, W1
`includes/class-ctgen-ajax-handlers.php`	Guard; size validation; image check; `make_subsize()` with fallback; translated messages; `authorize()`	S3, S7, S8, B1, W1, W5, W6
`includes/class-ctgen-functions.php`	Guard; `esc_html()` for the source column; translated strings	S6, S7, W1
`templates/ctgen-custom-thumb-list-html.php`	Valid rows; `esc_html()` ; reads validated sizes	S6, W2, W4
`templates/ctgen-form-html.php`	`max` / `step` on inputs; `__DIR__` include	S3, W3
`webp/class-ctgen-webp-ajax-handlers.php`	`delete_webp_copies()` on `delete_attachment`	S1
`webp/class-ctgen-webp-converter.php`	Stale temp-file cleanup	S5

`assets/js/ctgen-media-actions.js` (new), `.min.js` (rebuilt)	Readable source; text-only insertion; server messages; run never stalls; translatable	S2, S9, B2, B3, W1
`readme.txt`, `languages/custom-thumbnail-generator.pot`	WebP deletion documented; changelog; 21 new strings	-
`v2/` (not shipped)	`tests/cr-probe.php`, `tests/cr-browser.js` (new); `tests/m4-run-all.sh` runs them; `tests/m2-verify.php` calls the enqueue function as WordPress does; `tools/build-js.js` builds the third script; `deploy/svn-release.sh` checks the version constant	-

Backups of every edited file were kept next to it as `*.bak7` (excluded from the release by `.distignore`). Nothing was committed to version control.

10. Limitations

- This is a source review plus targeted dynamic testing on one local site (Chromium 130, PHP 7.4, Imagick); it is not a penetration test of a production server. Other browsers, multisite, object caches and media offloaded to S3/CDN were not tested.
- The fallback path for image editors without `make_subsize()` was tested with a test editor that wraps GD, not with a real third-party editor.
- The 25-10,000 px limits for new sizes are a design choice of this review (25 matches the existing form); sizes already stored are not affected.
- WebP deletion (S1) is intentionally conservative: a WebP copy whose dimensions no longer match its source (for example after the source was replaced by hand) is left in place.